# Manual de uso de C extended

Entregable 4 | Analizador léxico en Windows Forms

Este manual explica cómo abrir la aplicación, cargar o escribir código, analizarlo, consultar sus resultados y exportarlos. C-extended reconoce elementos léxicos de un subconjunto propio basado en C#; no ejecuta el programa ni sustituye una revisión sintáctica o semántica.

### Abrir la aplicación

Se necesita Windows con .NET Framework 4.8. Abra C-Extended.exe desde la carpeta de ejecución y conserve sus archivos acompañantes, incluido C-Extended.exe.config. El SDK usado para desarrollar o ejecutar pruebas no es un requisito para utilizar el ejecutable.

### Cargar y analizar código

1. Pulse Cargar archivo, elija un archivo .txt y confirme la apertura. Se admite UTF-8 y UTF-16 con BOM; si hay problemas de codificación, guarde el archivo nuevamente como UTF-8.

2. Revise el nombre de la fuente y el código del editor. También puede escribir o pegar código directamente. Los números del margen no forman parte del texto analizado.

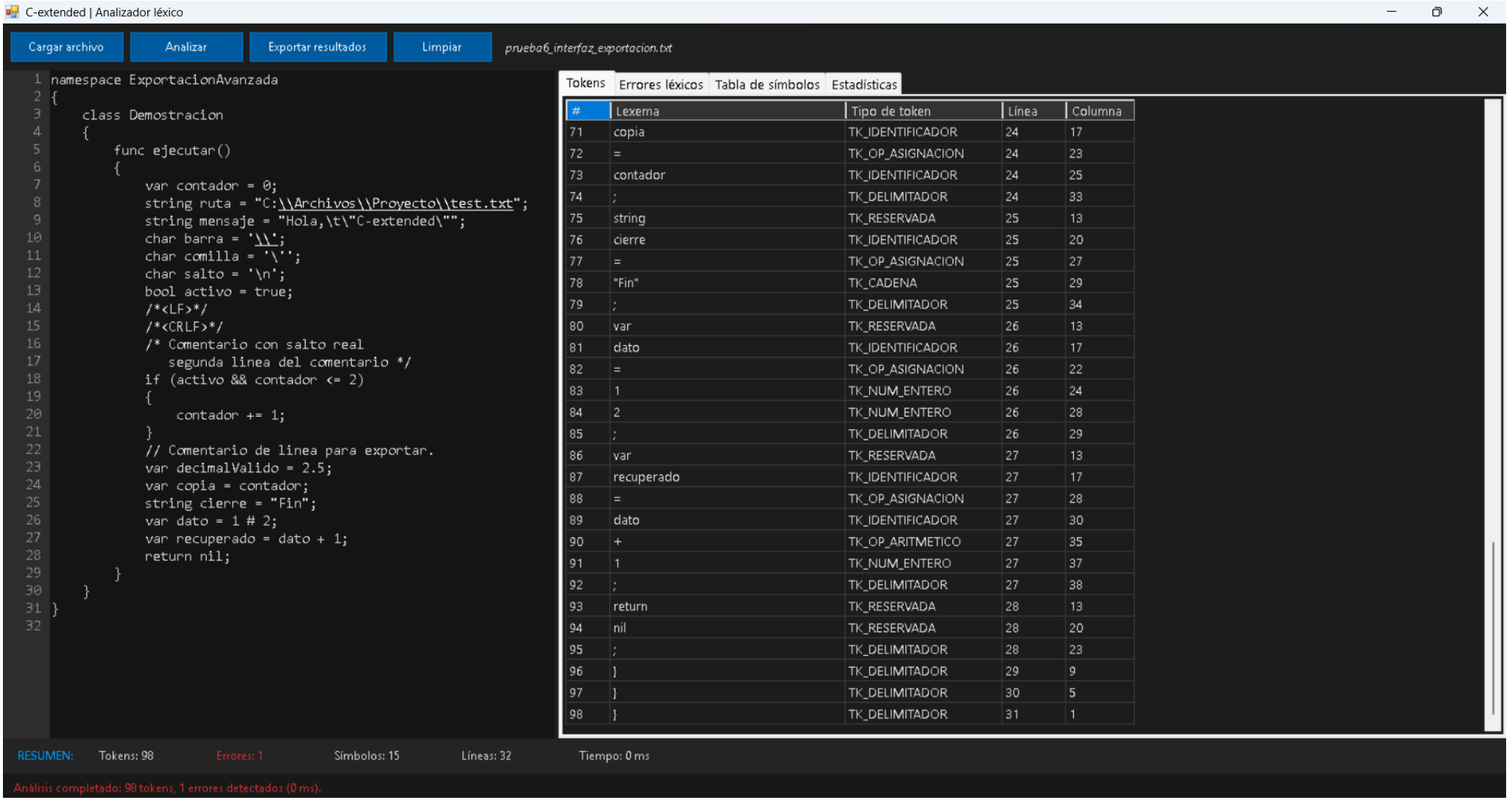
3. Pulse Analizar. Se procesa el contenido actual del editor. Consulte Tokens, Errores léxicos, Tabla de símbolos y Estadísticas; el resumen inferior muestra los totales.

| Botón | Acción |
|---|---|
| Cargar archivo | Selecciona y carga un archivo de código. Cancelar conserva la sesión actual. |
| Analizar | Reconoce los tokens y actualiza errores, símbolos y estadísticas. |
| Exportar resultados | Guarda un informe TXT del último análisis vigente. |
| Limpiar | Vacía el editor y los resultados, y deja la exportación deshabilitada. |

**Ejemplo de este manual:** use Entradas/Prueba6_nueva/prueba6_interfaz_exportacion.txt, incluido en la carpeta de documentación. Produce 98 tokens, 15 símbolos únicos, 1 error intencional y 32 líneas, incluida la línea vacía final.

## Lectura de tokens

Cada fila muestra el orden del token, su lexema, su categoría y la línea y columna donde comienza; las posiciones empiezan en 1. Use la barra de desplazamiento para recorrer la tabla. La captura muestra los últimos tokens: recuperado y return se reconocen después del error de la línea 26.




```
1  namespace ExportacionAvanzada
2  {
3      class Demostracion
4      {
5          func ejecutar()
6          {
7              var contador = 0;
8              string ruta = "C:\\Archivos\\Proyecto\\test.txt";
9              string mensaje = "Hola,\t\"C-extended\"";
10             char barra = '\\';
11             char comilla = '\'';
12             char salto = '\n';
13             bool activo = true;
14             /*<LF>*/
15             /*<CRLF>*/
16             /* Comentario con salto real
17                segunda linea del comentario */
18             if (activo && contador <= 2)
19             {
20                 contador += 1;
21             }
22             // Comentario de linea para exportar.
23             var decimalValido = 2.5;
24             var copia = contador;
25             string cierre = "Fin";
26             var dato = 1 # 2;
27             var recuperado = dato + 1;
28             return nil;
29         }
30     }
31 }
32
```

| # | Lexema | Tipo de token | Línea | Columna |
|---|---|---|---|---|
| 71 | copia | TK_IDENTIFICADOR | 24 | 17 |
| 72 | = | TK_OP_ASIGNACION | 24 | 23 |
| 73 | contador | TK_IDENTIFICADOR | 24 | 25 |
| 74 | ; | TK_DELIMITADOR | 24 | 33 |
| 75 | string | TK_RESERVADA | 25 | 13 |
| 76 | cierre | TK_IDENTIFICADOR | 25 | 20 |
| 77 | = | TK_OP_ASIGNACION | 25 | 27 |
| 78 | "Fin" | TK_CADENA | 25 | 29 |
| 79 | ; | TK_DELIMITADOR | 25 | 34 |
| 80 | var | TK_RESERVADA | 26 | 13 |
| 81 | dato | TK_IDENTIFICADOR | 26 | 17 |
| 82 | = | TK_OP_ASIGNACION | 26 | 22 |
| 83 | 1 | TK_NUM_ENTERO | 26 | 24 |
| 84 | 2 | TK_NUM_ENTERO | 26 | 28 |
| 85 | ; | TK_DELIMITADOR | 26 | 29 |
| 86 | var | TK_RESERVADA | 27 | 13 |
| 87 | recuperado | TK_IDENTIFICADOR | 27 | 17 |
| 88 | = | TK_OP_ASIGNACION | 27 | 28 |
| 89 | dato | TK_IDENTIFICADOR | 27 | 30 |
| 90 | + | TK_OP_ARITMETICO | 27 | 35 |
| 91 | 1 | TK_NUM_ENTERO | 27 | 37 |
| 92 | ; | TK_DELIMITADOR | 27 | 38 |
| 93 | return | TK_RESERVADA | 28 | 13 |
| 94 | nil | TK_RESERVADA | 28 | 20 |
| 95 | ; | TK_DELIMITADOR | 28 | 23 |
| 96 | } | TK_DELIMITADOR | 29 | 9 |
| 97 | } | TK_DELIMITADOR | 30 | 5 |
| 98 | } | TK_DELIMITADOR | 31 | 1 |

*Figura 1. Editor, tabla de tokens y resumen del caso de ejemplo. El carácter # se informa en la pestaña de errores y no se registra como token válido.*

## Lectura de errores léxicos

Esta pestaña indica la secuencia inválida, su línea y columna y la descripción del problema. En el ejemplo, # aparece en 26:26 como Carácter no permitido. El analizador continúa cuando la regla de recuperación lo permite; los tokens y símbolos recuperados siguen disponibles y el resultado puede exportarse.

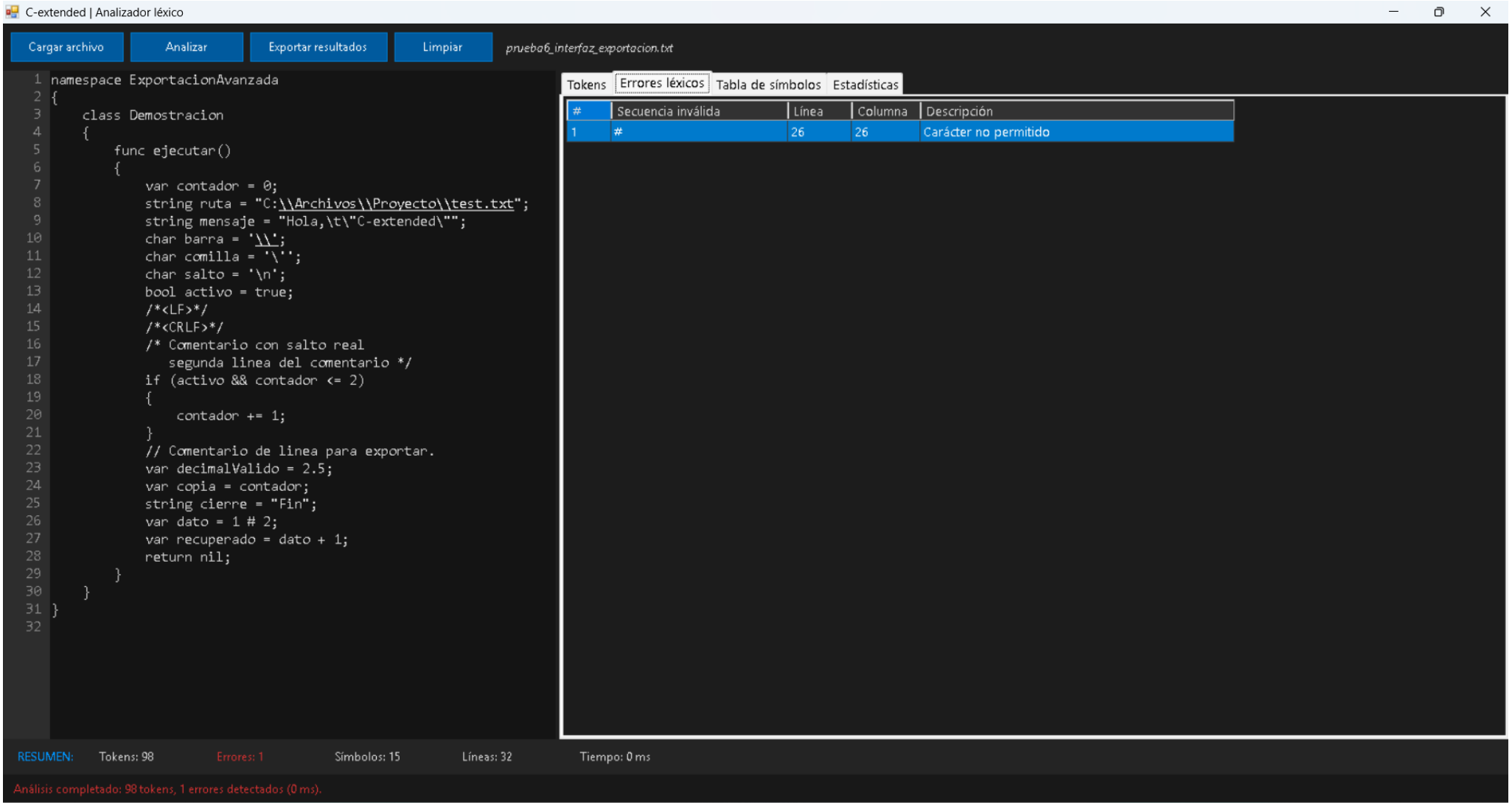
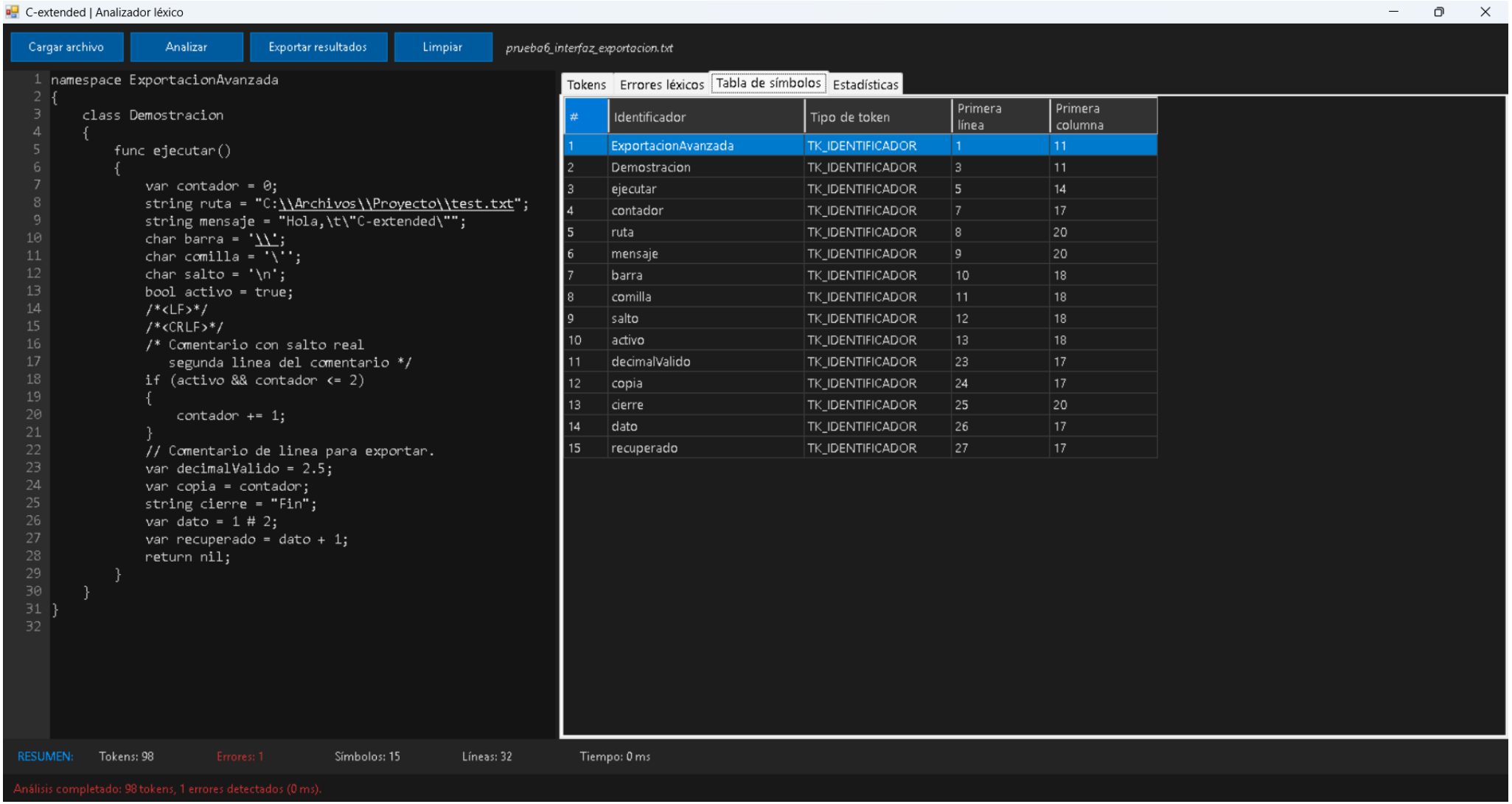



*Figura 2. Error intencional del caso de ejemplo. Si no se detectan errores, la aplicación muestra No se encontraron errores léxicos.*

## Consulta de la tabla de símbolos

La tabla reúne los identificadores únicos en orden de primera aparición. Aunque contador se usa varias veces, ocupa una sola fila y conserva su primera ubicación, 7:17. TK_IDENTIFICADOR expresa una categoría léxica; no es el tipo de dato ni el ámbito semántico. Se distinguen mayúsculas y minúsculas.




```
namespace ExportacionAvanzada
{
    class Demostracion
    {
        func ejecutar()
        {
            var contador = 0;
            string ruta = "C:\\Archivos\\Proyecto\\test.txt";
            string mensaje = "Hola,\t\"C-extended\"";
            char barra = '\\';
            char comilla = '\'';
            char salto = '\n';
            bool activo = true;
            /*<LF>*/
            /*<CRLF>*/
            /* Comentario con salto real
               segunda linea del comentario */
            if (activo && contador <= 2)
            {
                contador += 1;
            }
            // Comentario de linea para exportar.
            var decimalValido = 2.5;
            var copia = contador;
            string cierre = "Fin";
            var dato = 1 # 2;
            var recuperado = dato + 1;
            return nil;
        }
    }
}
```

| # | Identificador | Tipo de token | Primera línea | Primera columna |
|---|---|---|---|---|
| 1 | ExportacionAvanzada | TK_IDENTIFICADOR | 1 | 11 |
| 2 | Demostracion | TK_IDENTIFICADOR | 3 | 11 |
| 3 | ejecutar | TK_IDENTIFICADOR | 5 | 14 |
| 4 | contador | TK_IDENTIFICADOR | 7 | 17 |
| 5 | ruta | TK_IDENTIFICADOR | 8 | 20 |
| 6 | mensaje | TK_IDENTIFICADOR | 9 | 20 |
| 7 | barra | TK_IDENTIFICADOR | 10 | 18 |
| 8 | comilla | TK_IDENTIFICADOR | 11 | 18 |
| 9 | salto | TK_IDENTIFICADOR | 12 | 18 |
| 10 | activo | TK_IDENTIFICADOR | 13 | 18 |
| 11 | decimalValido | TK_IDENTIFICADOR | 23 | 17 |
| 12 | copia | TK_IDENTIFICADOR | 24 | 17 |
| 13 | cierre | TK_IDENTIFICADOR | 25 | 20 |
| 14 | dato | TK_IDENTIFICADOR | 26 | 17 |
| 15 | recuperado | TK_IDENTIFICADOR | 27 | 17 |

*Figura 3. Los 15 identificadores únicos del ejemplo, con su primera línea y columna. La lista incluye nombres de espacios, clases y funciones, además de variables.*

## Interpretación de las estadísticas

El panel presenta totales y distribución por categoría; los comentarios también cuentan como tokens. Las categorías del ejemplo suman 98. El conteo de líneas incluye las vacías y la línea final creada por un salto. El tiempo se muestra en milisegundos enteros: una entrada pequeña puede indicar 0 ms y haber sido analizada correctamente.

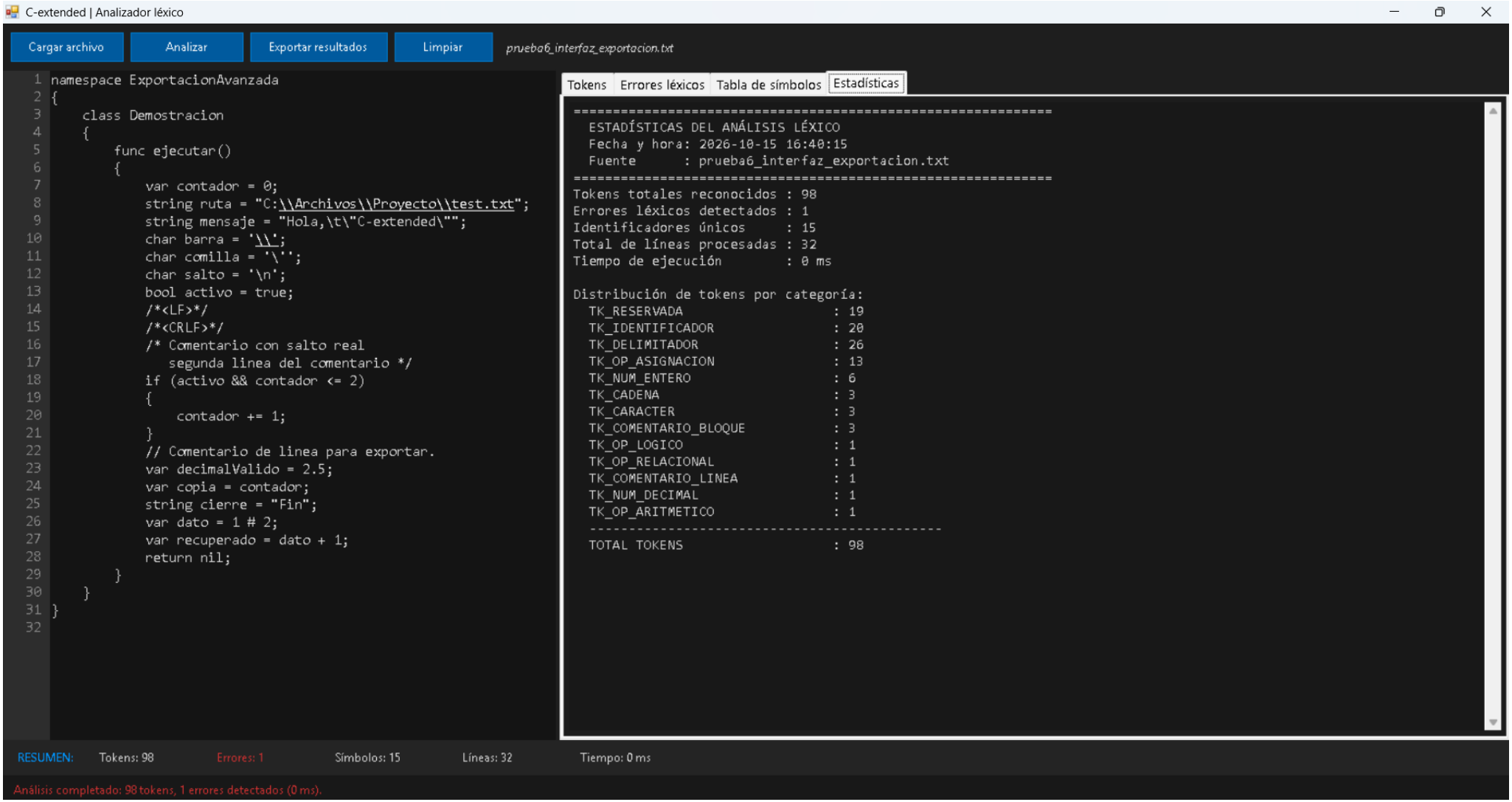



*Figura 4. Totales y distribución del mismo análisis. El tiempo y la fecha cambian entre ejecuciones; los resultados léxicos del archivo permanecen iguales.*

# Exportación y continuidad del trabajo

### Guardar los resultados

1. Tras analizar, pulse Exportar resultados. Elija una carpeta y un nombre terminado en .txt y pulse Guardar. Espere la confirmación de éxito; cancelar el diálogo conserva el análisis.

2. Abra el TXT guardado. Debe contener la fuente, fecha del análisis, fecha de exportación, resumen, categorías y tablas de tokens, errores y símbolos. Para este ejemplo, compruebe 98 tokens, 1 error (# en 26:26), 15 símbolos y 32 líneas.

3. Los saltos, tabulaciones, comillas y barras de los lexemas se muestran mediante escapes explicados en el encabezado. El informe guarda resultados; no reemplaza un archivo del código fuente escrito en el editor.

### Editar y volver a analizar

Al modificar el editor, las tablas y estadísticas se limpian y Exportar resultados se deshabilita. Pulse Analizar nuevamente para obtener resultados vigentes. Limpiar elimina el texto y los resultados de la sesión; conserve por separado el código que necesite reutilizar.

### Mensajes habituales

| Situación | Qué hacer |
|---|---|
| Editor vacío o solo espacios | Escriba o cargue código antes de pulsar Analizar. |
| Código modificado o exportación deshabilitada | Vuelva a analizar el contenido actual del editor. |
| Archivo con UTF-8 inválido, NUL o codificación no admitida | Revise el archivo y guárdelo como texto UTF-8. El rechazo de carga conserva el contenido anterior. |
| Error de acceso, lectura o escritura | Revise la ruta y sus permisos. Para exportar, pruebe una carpeta donde pueda guardar archivos. |
| Se detectaron errores léxicos | Consulte sus posiciones y corrija el código si corresponde. Puede exportar el diagnóstico antes de corregirlo. |

### Apoyo de herramientas de inteligencia artificial

Se utilizó Antigravity como apoyo a la implementación de la interfaz e integración del proyecto, y Codex para revisión, propuestas de corrección, comprobaciones y preparación de esta documentación. El grupo aplicó los cambios y comprobó los flujos de la aplicación. Las capturas corresponden al programa del grupo en ejecución.

Fuente: Proyecto_Analizador_Lexico_CSharp_WinForms, sección del Entregable 4, páginas 12 y 13, y normas generales sobre apoyo de IA, página 16.